

torch.autograd.functional.hessian#

<https://pytorch.org/docs/stable/generated/torch.autograd.functional.hessian>.

Description:

Compute the Hessian of a given scalar function. `func` (function) – a Python function that takes Tensor inputs and returns a Tensor with a single element. `inputs` (tuple of Tensors or Tensor) – inputs to the function `func`. `create_graph` (bool, optional) – If True, the Hessian will be computed in a differentiable manner. Note that when `strict` is False, the result can not require gradients or be disconnected from the inputs. Defaults to False. `strict` (bool, optional) – If True, an error will be raised when we detect that there exists an input such that all the outputs are independent of it. If False, we return a Tensor of zeros as the hessian for said inputs, which is the expected mathematical value. Defaults to False.

Function Signature

```
torch.autograd.functional.hessian(func, inputs,  
create_graph=False, strict=False, vectorize=False,  
outer_jacobian_strategy='reverse-mode')[source]#
```

Parameters

Returns

Return type

Returns:

Hessian (Tensor or a tuple of tuple of Tensors)

Code Examples

```
>>> def pow_reducer(x):
...     return x.pow(3).sum()
>>> inputs = torch.rand(2, 2)
>>> hessian(pow_reducer, inputs)
tensor([[[[5.2265, 0.0000],
          [0.0000, 0.0000]],
        [[0.0000, 4.8221],
          [0.0000, 0.0000]]],
       [[0.0000, 0.0000],
        [1.9456, 0.0000]],
       [[0.0000, 0.0000],
        [0.0000, 3.2550]]]])
```

```
>>> hessian(pow_reducer, inputs, create_graph=True)
tensor([[[[5.2265, 0.0000],
          [0.0000, 0.0000]],
        [[0.0000, 4.8221],
          [0.0000, 0.0000]]],
       [[0.0000, 0.0000],
        [1.9456, 0.0000]],
       [[0.0000, 0.0000],
        [0.0000, 3.2550]]]], grad_fn=<ViewBackward>)
```

```

>>> def pow_adder_reducer(x, y):
...     return (2 * x.pow(2) + 3 * y.pow(2)).sum()
>>> inputs = (torch.rand(2), torch.rand(2))
>>> hessian(pow_adder_reducer, inputs)
(tensor([[4., 0.],
         [0., 4.]]),
 tensor([[0., 0.],
         [0., 0.]])),
(tensor([[0., 0.],
         [0., 0.]]),
 tensor([[6., 0.],
         [0., 6.]]))

```

Detailed Documentation

Documentation

Compute the Hessian of a given scalar function.

func (function) – a Python function that takes Tensor inputs and returns a Tensor with a single element.

inputs (tuple of Tensors or Tensor) – inputs to the function func.

create_graph (bool, optional) – If True, the Hessian will be computed in a differentiable manner. Note that when strict is False, the result can not require gradients or be disconnected from the inputs. Defaults to False.

strict (bool, optional) – If True, an error will be raised when we detect that there exists an input such that all the outputs are independent of it. If False, we return a Tensor of

zeros as the hessian for said inputs, which is the expected mathematical value. Defaults to False.

`vectorize` (bool, optional) – This feature is experimental. Please consider using `torch.func.hessian()` instead if you are looking for something less experimental and more performant. When computing the hessian, usually we invoke `autograd.grad` once per row of the hessian. If this flag is True, we use the `vmap` prototype feature as the backend to vectorize calls to `autograd.grad` so we only invoke it once instead of once per row. This should lead to performance improvements in many use cases, however, due to this feature being incomplete, there may be performance cliffs. Please use `torch._C._debug_only_display_vmap_fallback_warnings(True)` to show any performance warnings and file us issues if warnings exist for your use case. Defaults to False.

`outer_jacobian_strategy` (str, optional) – The Hessian is computed by computing the Jacobian of a Jacobian. The inner Jacobian is always computed in reverse-mode AD. Setting strategy to "forward-mode" or "reverse-mode" determines whether the outer Jacobian will be computed with forward or reverse mode AD. Currently, computing the outer Jacobian in "forward-mode" requires `vectorized=True`. Defaults to "reverse-mode".

if there is a single input, this will be a single Tensor containing the Hessian for the input. If it is a tuple, then the Hessian will be a tuple of tuples where `Hessian[i][j]` will contain the Hessian of the *i*th input and *j*th input with size the sum of the size of the *i*th input plus the size of the *j*th input. `Hessian[i][j]` will have the same dtype and device as the corresponding *i*th input.

Hessian (Tensor or a tuple of tuple of Tensors)